



NETWORK SECURITY | MODULE 6

Secret Key Cryptography and DES

Lesson 10 – Secret Key Cryptography

Lesson 12 – The Data Encryption Standard (DES)



Symmetric keys · KDC · Vernam · Book cipher · MAC · Feistel · S-boxes · Avalanche



LESSON 10: SECRET KEY CRYPTOGRAPHY

1

Four Aspects of Security

Privacy, authentication, integrity, non-repudiation

2

Secret-Key Encryption

One shared key, pros, cons and the KDC

3

Classical Secret-Key Ciphers

Columnar transposition, Vernam, book cipher

4

Algorithm, Key and Integrity

Combination lock, checksums and MACs

LESSON 12: DATA ENCRYPTION STANDARD

5

DES Structure

64-bit blocks, 56-bit key, 16 rounds

6

Inside a Round

Feistel, function F, S-boxes, subkeys

7

Strength of DES

Avalanche, 56-bit keys, timing attacks

8

DES Design Criteria

Rounds, F, S-box design, key schedule



Lesson 10

Secret Key Cryptography

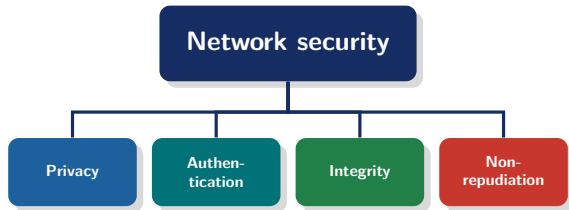


Fig. 10.1: Aspects of security

As more data travels over the Internet, users expect to:

- keep messages **confidential**
- be sure of **data integrity**
- **identify** the sender of a message
- **prove** a message really came from a sender, even if the sender denies it

So security has four aspects

Privacy, authentication, integrity and non-repudiation.

The Four Aspects Explained



Privacy

The message makes sense **only to the intended receiver**; to all others it is unintelligible

Authentication

The receiver is sure of the **sender's identity**, not an imposter

Integrity

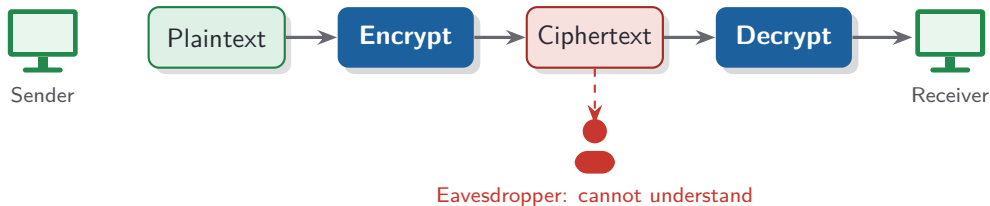
Data arrives **exactly as sent**. A \$10,000 transfer must not become \$100,000

Non-repudiation

The receiver can **prove** who sent it, e.g. a bank proving the customer asked for a transfer

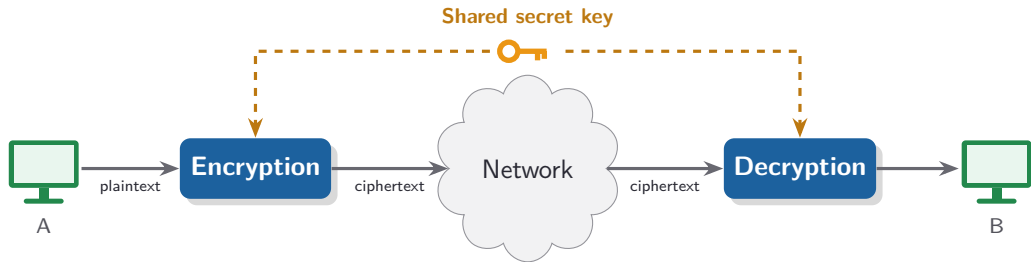
The burden of proof for non-repudiation falls on the **receiver**.

Privacy: Plaintext and Ciphertext



- The concept of privacy **has not changed for thousands of years**: encrypt the message
- Data at the sender is **plaintext**; encrypted data is **ciphertext**, decrypted at the receiver
- Two categories of methods: **secret-key** methods and **public-key** methods

Secret-Key Encryption (Fig. 10.2)



Key idea

The **same key** is used by the sender (to encrypt) and the receiver (to decrypt). The key is **shared**.

- Decryption is the **inverse** of encryption: if encryption adds and multiplies, decryption divides and subtracts
- Also called **symmetric** encryption: the same key works in both directions

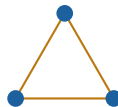


Advantage: efficiency

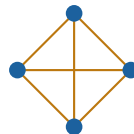
Secret-key algorithms take **less time** to encrypt than public-key ones, because the key is usually smaller. So they are used for **long messages**.

Disadvantages

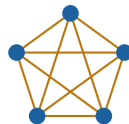
1. Each **pair** of users needs its own secret key
2. **Distributing** keys between parties can be difficult



3 users: 3 keys



4 users: 6 keys

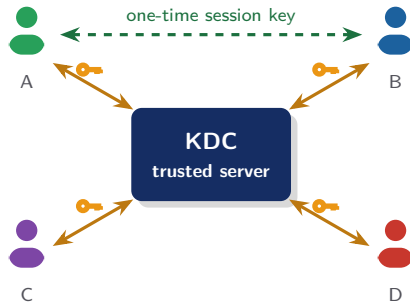


5 users: 10 keys

$$N \text{ users need } \frac{N(N-1)}{2} \text{ keys}$$

1 million people $\Rightarrow$ about **500 billion** keys

Key Distribution Center (KDC)



- Two parties who may **never meet** must still agree on a shared secret key
- Solution: both **trust a third party**, the KDC, which shares one key with **each user**
- Users use their KDC key to set up a **one-time shared key** with each other

Boxed rule

KDC can solve the problem of secret key distribution.

Columnar Transposition: Round 1



Plaintext: **come home tomorrow**

C1	C2	C3	C4	C5	C6
c	o	m	e	h	o
m	e	t	o	m	o
r	r	o	w		

read order



- 1 Write the message **row by row** in a rectangle of 6 columns
- 2 Choose a random column order, e.g. **4 6 1 2 5 3**
- 3 Read the text **column by column** in that order

Ciphertext after round 1

eow oo cmr oer hm mto

Columnar Transposition: Multiple Rounds (Fig. 10.3)



Round 2 input: **eowoo cmroerhmmto**

C1	C2	C3	C4	C5	C6
e	o	w	o	o	c
m	r	o	e	r	h
m	m	t	o		
3	4	6	1	5	2

- Repeat steps 1–3 on the round-1 ciphertext
- Use the **same column order** 4 6 1 2 5 3
- Continue for more rounds if desired

Ciphertext after round 2

oeo ch emm orm or wot

Why repeat?

Multiple rounds make the ciphertext **much harder to crack** than one round.

Vernam Cipher (One-Time Pad)

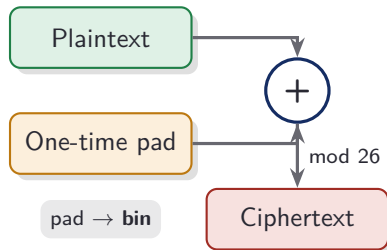


Idea

Use a **random, non-repeating** set of characters as the key (the pad). The pad is as long as the plaintext and is **never used again**, hence “one-time”.

Algorithm (Fig. 10.4)

1. Number letters: A=0, B=1, ..., Z=25
2. Do the same for each pad letter
3. **Add** plaintext and pad numbers
4. If the sum > 25 , subtract 26
5. Turn the numbers back into letters



Vernam Cipher: Worked Example (Fig. 10.5)



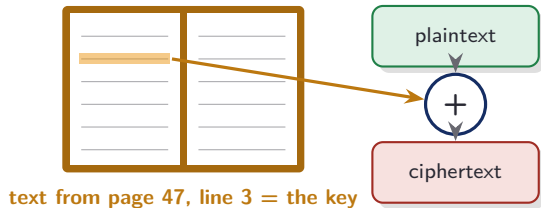
Plaintext	H	O	W	A	R	E	Y	O	U
	7	14	22	0	17	4	24	14	20
One-time pad	N	C	B	T	Z	Q	A	R	X
	13	2	1	19	25	16	0	17	23
Sum	20	16	23	19	42	20	24	31	43
If > 25, -26	20	16	23	19	16	20	24	5	17
Ciphertext	U	Q	X	T	Q	U	Y	F	R

Strength

Pad discarded after one use, so it is **highly secure**. First built at **AT&T** as the Vernam Machine.

Weakness

A pad as long as the message, used once, is **impractical for large messages**. Suitable only for short ones.

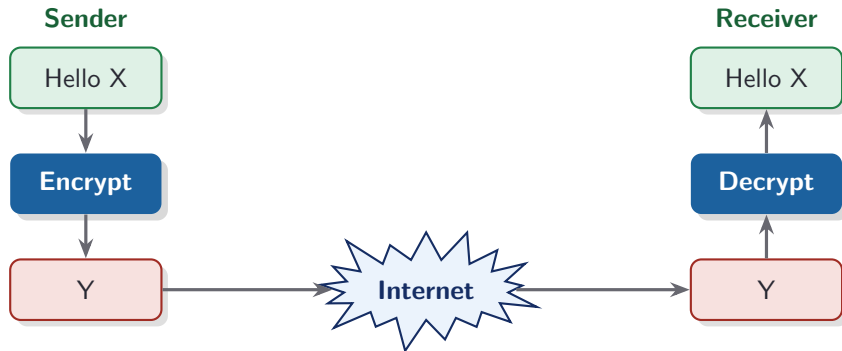


- Quite simple, and **similar in principle** to the Vernam cipher
- The “pad” is a portion of text from a **book**
- Book characters are **added** to the plaintext, just as with a one-time pad

Practical advantage

Both parties only need to agree on a **book and a position**, instead of exchanging a long random pad.

Encryption and Decryption in the Real World (Fig. 10.8)



Encryption

Transforms **plaintext** into **ciphertext** at the sender's computer.

Decryption

The **exact opposite**: ciphertext back to plaintext at the receiver's computer.



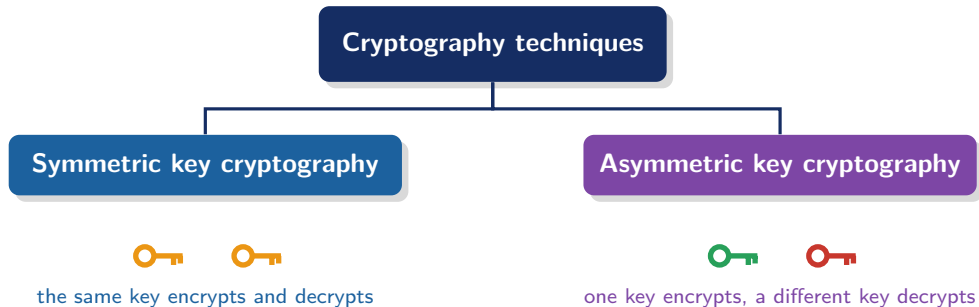
LOCK ← Algorithm

← Key

- The decryption algorithm must **match** the encryption algorithm, or the plaintext cannot be recovered
- The **key** is like the one-time pad: only sender and receiver know it
- A combination lock: everyone knows **how** it opens (algorithm), but only you know **871** (key)

The golden rule

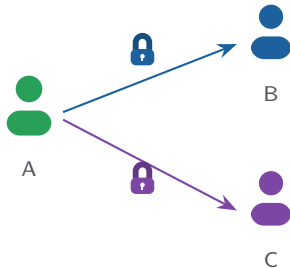
The algorithm is usually **public**. It is the **secret key** that makes cryptography secure.



This module

Lesson 10 and DES are **symmetric**. Asymmetric (public-key) cryptography comes in **Module 8**.

The Lock-and-Key Problem (Fig. 10.13)

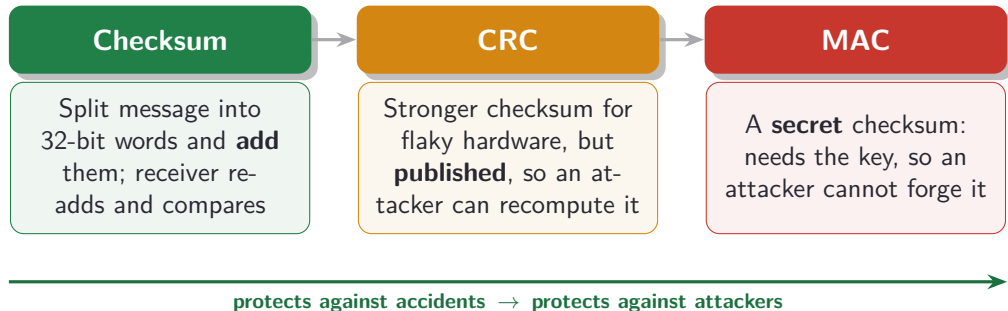


A needs a **different** lock and key for B and for C

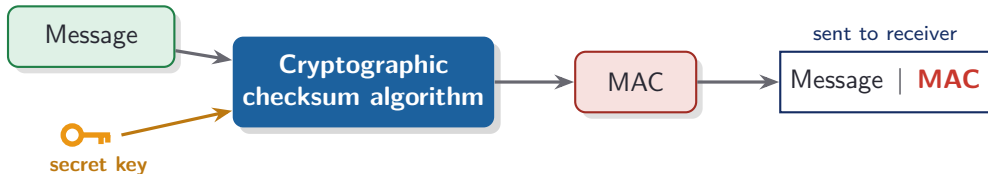
Parties	Lock-and-key pairs
2 (A, B)	1
3 (A, B, C)	3
4 (A, B, C, D)	6
5 (A, B, C, D, E)	10

If B and C had the same key, B could open C's box. So every pair needs its **own** key.

Integrity Check: From Checksum to MAC



Message Authentication Code (MAC)



- As with encryption: a **known algorithm** plus a **secret key**
- Also called a **MIC** (message integrity code); MAC is the more popular term
- Used for years to protect large **interbank fund transfers**: not secret, but integrity is ensured

How hard to forge?

A typical MAC is at least **48 bits**, so the chance of guessing it is about **1 in 280 trillion**.



Q1

How many keys do 10 people need with secret-key cryptography?

$$10 \times 9 \div 2 = \mathbf{45} \text{ keys}$$

Q2

Encrypt the letter H with pad letter N in the Vernam cipher.

$$7 + 13 = 20 \Rightarrow \mathbf{U}$$

Q3

Who solves the key distribution problem?

A Key Distribution Center (KDC)

Q4

Why is a MAC better than a CRC against attackers?

A MAC needs the **secret key**; a CRC can be recomputed by anyone

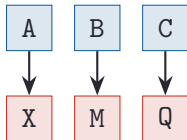


Lesson 12

Data Encryption Standard

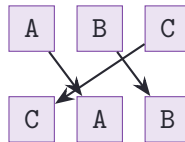


Substitution $\Rightarrow$ Confusion



A symbol is **replaced** by another. This hides the relationship between key and ciphertext.

Transposition $\Rightarrow$ Diffusion

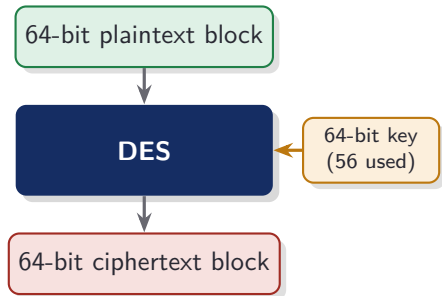


The **order** is rearranged. This spreads each plaintext bit over many ciphertext bits.

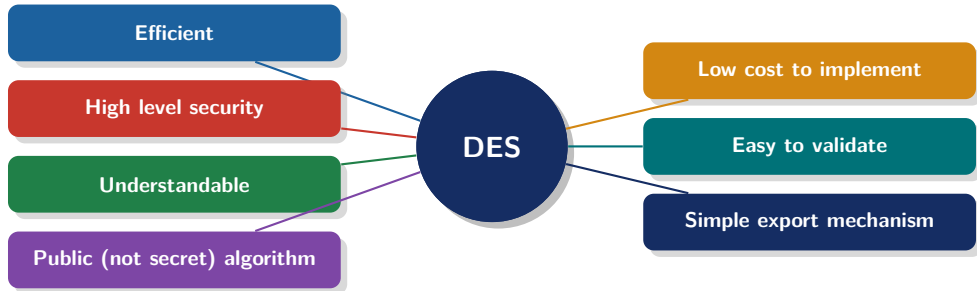
Product ciphers

Alternate substitution and transposition phases to get **both** confusion and diffusion. DES is such a product cipher.

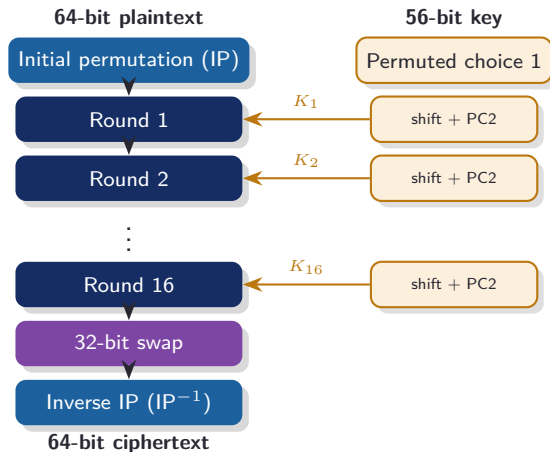
What Is DES?



- Also called the **Data Encryption Algorithm (DEA)**
- Selected as an official US **FIPS** standard in **1976**; then used worldwide
- The archetypal **block cipher**: block size **64 bits**
- Key is 64 bits, but **8 are parity bits**, so the effective key is **56 bits**
- Intense study of DES shaped the modern understanding of block ciphers



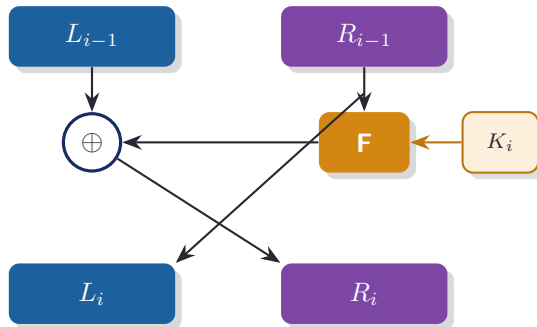
Overall Structure of DES (Fig. 12.1)



- 1 **Initial permutation** rearranges the 64 plaintext bits
- 2 **16 rounds** of the same function (permutation + substitution)
- 3 Left and right halves are **swapped**
- 4 **Inverse IP** produces the ciphertext

On the right, the key passes through a permutation, then each round gets a **different subkey K_i** from **left circular shifts** and a permutation.

One Round: The Feistel Structure



Each 64-bit value is split into two 32-bit halves, **L** and **R**. Each round computes:

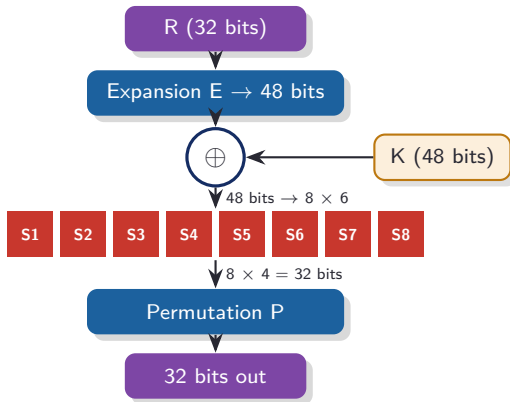
Feistel formulas

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

$\oplus$ is bitwise XOR. Apart from the initial and final permutations, DES has **exactly the structure of a Feistel cipher**.

Inside the Function F (Fig. 12.3)



- 1 The 32-bit R is **expanded** to 48 bits (16 bits duplicated)
- 2 XORed with the 48-bit round key K_i
- 3 Passed through **8 S-boxes**: each takes 6 bits and outputs 4
- 4 The 32-bit result is **permuted** by P

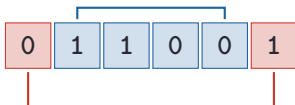
The S-boxes

The only **nonlinear** part of DES, and the heart of its security.

How to Read an S-Box: Worked Example



column = middle 4 = 1100 = 12



row = first+last = 01 = 1

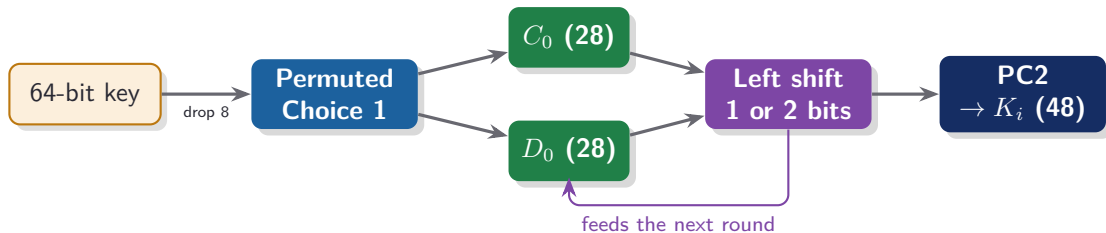
S_1 row 1, column 12 = 9
output = 1001

- Each S-box has **4 rows** and **16 columns**
- The **first and last** input bits pick the row (0–3)
- The **middle four** bits pick the column (0–15)
- The number in that cell, written in 4 bits, is the output
- Each row is a **reversible** substitution

Example from the SLM

Input 011001 to S_1 gives output 1001.

The Key Schedule

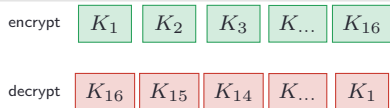


- Bits are numbered 1–64; **every eighth bit is ignored**, leaving 56
- The 56 bits are split into two 28-bit halves, C_0 and D_0
- Each round, both halves rotate left by **1 or 2 bits**; Permuted Choice 2 then picks the **48-bit** subkey for $F(R_{i-1}, K_i)$



Decryption

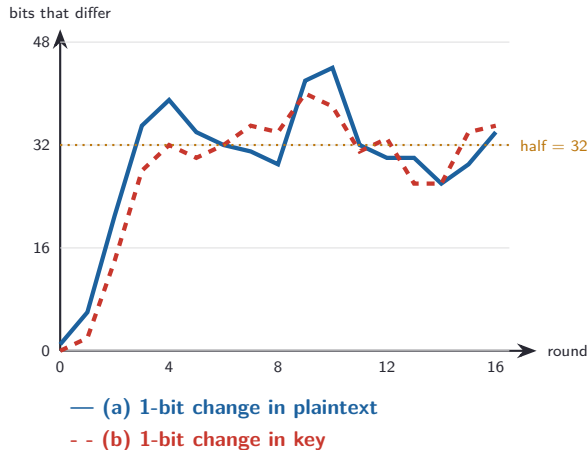
Same algorithm as encryption, with the subkeys in **reverse order**: K_{16} first, then $K_{15}, \dots$, and K_1 last.



Characteristics of DES

- 1 Each ciphertext bit depends on **all** key bits and **all** plaintext bits
- 2 No **statistical relationship** between plaintext and ciphertext
- 3 Changing one plaintext or key bit changes the ciphertext greatly (**avalanche effect**)
- 4 Changing ciphertext bits gives an **unpredictable** change in the recovered plaintext

The Avalanche Effect (Table 12.1)



Definition

A small change in the plaintext or the key should produce a **significant change** in the ciphertext.

- Plaintexts differing by 1 bit: after **3 rounds**, 21 bits differ; at the end, **34**
- Keys differing by 1 bit: about **half** the ciphertext bits differ after a few rounds



Type of analysis	Order of search	Type of attack
Exhaustive key search	$O(2^{56})$	Key attack
Differential cryptanalysis	$O(2^{47})$	Chosen plaintext
Linear cryptanalysis	$O(2^{43})$	Known plaintext

DES strength has been analysed since **1975**. The main worry has always been the **key size**: can 56 bits really provide security?

Exhaustive Search: How DES Fell



Diffie & Hellman: a \$20M parallel machine of 1 million chips could break DES in about **10 hours**

1977

EFF's "**DES cracker**", built for under \$250,000, breaks DES in **under 3 days**

July 1998

1990s

Wiener: a machine at one-twentieth of that price cracks DES in **3.5 hours**

Today

Agencies crack DES in a few hours; DES is **virtually worthless** alone



1 decryption per μs : **over 1000 years**
(searching half the key space)

- At first sight, brute force on 2^{56} keys seems **impractical**
- But one decryption per microsecond is a very **conservative** assumption
- Hardware keeps getting cheaper and faster, as the EFF showed

It is more than trying keys

The attacker must **recognize** the correct plaintext. Easy for English text; harder if the data was **compressed** or is numerical.



design criteria **not made public**

- Concern: cryptanalysis may exploit the **characteristics of the algorithm**
- The focus is the **eight S-boxes** used in each iteration
- Since their design was secret, people suspected a hidden weakness known to an opponent
- Many regularities and odd behaviours have been found

But...

No one has yet found the supposed **fatal weakness** in the S-boxes.



different inputs take slightly different times $\Rightarrow$ clues about the key

Definition

Information about the key or plaintext is found by **measuring how long** an implementation takes to decrypt different ciphertexts.

- Attacks the **implementation**, not the algorithm
- Timing data is **easier to get** than mathematical flaws
- Defence: make every call take the **same time**, at the cost of slower response
- DES appears fairly resistant to timing attacks



1

No output bit should be close to a **linear** function of the inputs

2

Each row must include all **16** possible outputs

3

Inputs differing in **1 bit** $\Rightarrow$ outputs differ in at least 2 bits

4

Inputs differing in the **2 middle bits** $\Rightarrow$ outputs differ in at least 2 bits

5

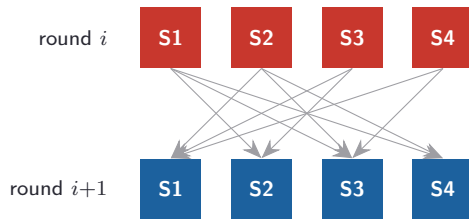
Inputs differing in the first 2 bits, same last 2 $\Rightarrow$ outputs **not the same**

6

For any nonzero 6-bit difference, at most **8 of 32** pairs share an output difference

Why criterion 1 matters most

The S-boxes are the **only nonlinear** part of DES. If they were linear, the whole cipher would be linear and **easily broken** (like the Hill cipher).



- 1 The 4 output bits of each S-box in round i : **two** feed “middle bits” and **two** feed “end bits” of round $i+1$
- 2 The 4 outputs of each S-box affect **six different** S-boxes next round; no two affect the same one
- 3 If an output of S_j hits a middle bit of S_k , no output of S_k may hit a middle bit of S_j

Goal

Increase the **diffusion** of the algorithm.



Number of rounds

Function F

Key schedule algorithm

three sources of a Feistel cipher's strength

Rounds

More rounds make cryptanalysis harder, even with a weak F. Choose enough rounds that known attacks need **more effort than brute force**. DES followed this rule.

Design of F

F is the **heart** of a Feistel cipher; in DES it relies on S-boxes. F provides **confusion**, so it must be hard to “unscramble”.

- F must be **nonlinear**: the harder to approximate with linear equations, the better
- **Bit Independence Criterion (BIC)**: output bits j and k change independently when any input bit i flips



- Any input change should give a **random-looking** output change
- An $n \times m$ S-box has n inputs and m outputs; DES uses 6×4
- Larger S-boxes resist cryptanalysis better, but the table grows **exponentially**; n is usually limited to 8–10

Key-dependent S-boxes

Random and key-based, so they cannot be analysed **in advance**.

Nyberg's four approaches

Random

Pseudorandom numbers; fine for large boxes

Random + testing

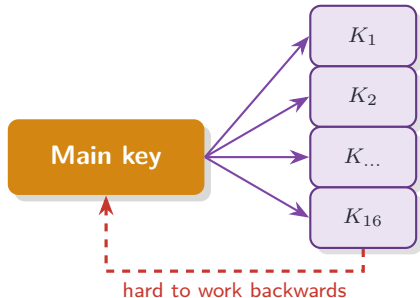
Keep only boxes that pass tests

Human-made

Manual design; used for DES

Math-made

Proven resistance to attacks



- Has received **less attention** than S-box design
- With any Feistel cipher, the key generates **one subkey per round**
- Subkeys should make it hard to **deduce individual subkeys**, and hard to work back to the **main key**

Minimum requirement

Guarantee the **Strict Avalanche Criterion** and the **Bit Independence Criterion** for key/ciphertext.



Feature	DES
Type	Symmetric block cipher (Feistel structure)
Block size	64 bits
Key size	64 bits, of which 56 are used (8 parity bits)
Rounds	16, each with its own 48-bit subkey
Nonlinear part	8 S-boxes, each $6 \rightarrow 4$ bits
Decryption	Same algorithm, subkeys in reverse order
Main weakness	Short key: broken by brute force (EFF, 1998)



Q1

What are the DES block size and effective key size?

64-bit block, 56-bit key

Q2

Write the Feistel formula for R_i .

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

Q3

How many S-boxes does DES use, and what size?

8 S-boxes, each 6 → 4 bits

Q4

How does DES decryption differ from encryption?

Same algorithm, subkeys in **reverse order**

Q5

Who broke DES by brute force in 1998?

The **EFF** “DES cracker”, in under 3 days



Lesson 10: Secret Key Cryptography

- Privacy, authentication, integrity, non-repudiation
- One shared key; $N(N-1)/2$ keys; the KDC
- Columnar transposition with rounds
- Vernam (one-time pad) and book cipher
- Algorithm vs. key; symmetric vs. asymmetric
- Checksum, CRC and MAC

Lesson 12: DES

- Confusion, diffusion, product ciphers
- 64-bit block, 56-bit key, 16 Feistel rounds
- Function F, S-boxes, key schedule
- Avalanche effect; strength; timing attacks
- Design criteria for S-boxes, P, rounds, F

Review questions

1. Describe the features of secret key cryptography. 2. Explain methods of privacy implementation. 3. Explain the Vernam cipher. 4. Discuss the book cipher. 5. Explain encryption and decryption in detail. 6. Write a short note on integrity check. 7. Describe the overall structure of DES. 8. Explain the avalanche effect in DES. 9. Discuss the DES design criteria.



- ① William Stallings, *Cryptography and Network Security*, PHI Publishers.
- ② Charlie Kaufman, Radia Perlman, *Network Security*, Pearson Education.
- ③ Bruce Schneier, *Applied Cryptography*, John Wiley & Sons.
- ④ Roberta Bragg, *Network Security: The Complete Reference*, TMH.

Thank You! Questions?